

你现在是一名资深 AI Agent 架构师、Python/TypeScript 全栈工程师、数据库工程师、RAG 工程师、安全工程师和大型项目重构专家。

请对当前仓库中的 **PrivateAgent** 项目进行一次完整、渐进、可验证的现代化改造。

本次任务不是简单更换几个依赖、修改页面样式或为了使用新技术而推倒重写，而是要在尽量保留现有核心功能、用户数据、会话记录、知识库和可复用代码的前提下，将 PrivateAgent 逐步重构为一个架构清晰、可扩展、可测试、可观测、可恢复并具备现代 Agent 能力的个人智能体平台。

一、总体目标

将当前 PrivateAgent 改造成一个现代化 Agent 系统，目标能力包括：

- 使用 LLM 作为理解、推理、规划和决策核心
- 使用 Agent Runtime 负责目标执行、工具循环、状态管理和任务终止判断
- 使用模型原生结构化工具调用，不通过正则表达式解析模型自然语言
- 支持普通 Function Tool
- 支持 MCP Tool 和 MCP Server
- 支持短期会话记忆
- 支持长期结构化记忆
- 支持语义记忆
- 支持动态上下文组装
- 支持 Token 预算控制
- 支持长会话压缩
- 支持任务摘要
- 支持记忆提取、去重和冲突处理
- 支持 RAG 知识库
- 支持关键词检索、向量检索和后续混合检索
- 支持来源引用
- 支持任务验证
- 支持受控重试和反思机制
- 支持流式输出
- 支持工具执行状态展示
- 支持高风险操作人工审批
- 支持任务取消
- 支持 Agent 运行轨迹和可观测性
- 为后续接入 LangGraph 的复杂工作流预留扩展能力
- 保留当前项目中仍然合理、稳定、可维护的模块
- 不为追求所谓“主流技术”而进行没有业务价值的重写

优先使用成熟、官方维护、当前稳定的技术方案。

不得根据旧教程使用已经废弃的 API、接口或设计模式。

新增依赖必须：

- 说明引入原因
- 锁定具体版本
- 检查与现有依赖的兼容性
- 更新依赖锁文件

- 说明是否存在升级风险

二、工作原则

必须遵循以下原则：

1. 首先完整阅读项目，不要立即开始大规模修改。
2. 先识别现有技术栈、目录结构、启动方式和核心数据流。
3. 先判断哪些模块应该保留、封装、替换或删除。
4. 不允许一次性推倒重写整个项目。
5. 使用小步、可验证、可回滚的方式实施改造。
6. 每完成一个阶段，都必须运行测试、静态检查、构建检查或启动验证。
7. 不要为了使用 LangChain、LangGraph、MCP 或向量数据库而强行增加抽象。
8. 简单的项目内部能力优先使用普通 Function Tool。
9. 需要独立部署、跨客户端复用、跨进程访问或连接外部系统的能力，再考虑设计为 MCP Server。
10. LangGraph 只在存在明确的条件分支、循环、人工审批、任务恢复、长任务或多 Agent 协作需求时引入。
11. 不要同时维护两套重复的 Agent 执行框架。
12. 不得删除现有用户数据、会话记录、知识库、记忆、工具配置或其他业务数据。
13. 涉及数据库结构修改时，必须提供迁移脚本、数据校验和恢复方案。
14. 涉及文件写入、Shell、部署、消息发送、删除数据、系统配置和数据库写操作时，必须增加权限控制。
15. 高风险操作必须经过人工确认。
16. 不得把 API Key、密码、Token 或数据库凭证写死在源码中。
17. 不得为了看起来架构先进而创建大量没有实际用途的空目录、空接口和抽象层。
18. 所有设计都必须能够在当前项目中运行，而不是只停留在文档或伪代码层面。
19. 所有结论必须引用具体文件、模块、类、函数、配置项或运行结果。
20. 如果现有实现合理，应优先兼容和复用，而不是无条件替换。

三、第一阶段：项目审计

请先完成项目审计，并生成：

```
docs/modernization-audit.md
docs/migration-plan.md
docs/target-architecture.md
```

在完成审计前，不要进行大规模重构。

1. 当前项目结构

分析并记录：

- 前端框架
- 后端框架
- 编程语言
- Python、Node.js 或其他运行时版本
- LLM 调用入口
- Agent 入口
- API 路由

- 前后端通信方式
- 流式输出方式
- 数据库
- ORM
- 数据库驱动
- 数据迁移工具
- 会话管理
- 工具系统
- 插件系统
- MCP 实现
- 记忆模块
- 上下文管理
- RAG 或知识库模块
- Embedding 模型
- 向量存储
- 文件处理
- PDF 解析
- 用户认证
- 权限管理
- 日志系统
- 配置管理
- Docker
- Docker Compose
- 部署方式
- 测试覆盖情况
- CI/CD 配置

2. 当前 Agent 执行链路

画出当前完整数据流：

用户输入
→ 前端
→ 后端接口
→ 模型调用
→ 工具选择
→ 工具执行
→ 结果返回模型
→ 数据存储
→ 最终输出

重点判断：

- 当前项目是否只是普通聊天应用
- 是否真正具备 Agent 工具循环
- 是否存在规划和执行阶段
- 是否支持模型多次调用工具
- 工具结果是否会重新进入模型上下文
- 是否能够判断任务已经完成
- 是否存在无限循环风险

- 是否有最大执行步数
- 是否有超时控制
- 是否支持任务取消
- 是否支持失败重试
- 是否记录 Agent 运行状态
- 是否展示工具执行过程
- 是否泄露模型隐藏推理过程
- 是否支持人工审批
- 是否能从中断状态恢复

3. 过时或不合理技术

列出所有发现的问题，并按等级分类：

- P0：安全问题、数据风险、无法运行或可能导致数据损坏的问题
- P1：核心 Agent 架构问题
- P2：严重影响扩展、性能和维护的问题
- P3：代码质量、文档和用户体验问题

每个问题必须说明：

- 问题名称
- 文件位置
- 当前实现
- 存在的问题
- 推荐方案
- 是否必须立即修改
- 修改风险
- 影响范围
- 验证方式

4. 技术处理清单

明确输出以下四类：

保留

当前实现合理，只需要补充测试、文档或少量优化。

封装

当前实现可用，但需要统一接口或兼容层。

替换

当前实现已经过时、风险过高或无法支持目标能力，需要渐进替换。

删除

当前代码重复、失效、无调用或明显不安全，可以在确认依赖关系后移除。

不要默认旧代码全部无价值。

四、目标后端架构

后端优先使用 Python，并根据现有项目实际情况决定是否继续沿用当前框架，或渐进式迁移到 FastAPI。

建议目标结构如下，但必须根据实际项目调整，不要机械照搬：

```
backend/
├── app/
│   ├── api/
│   │   ├── chat.py
│   │   ├── agents.py
│   │   ├── runs.py
│   │   ├── tasks.py
│   │   ├── tools.py
│   │   ├── approvals.py
│   │   ├── mcp.py
│   │   ├── memories.py
│   │   └── knowledge.py
│   ├── agents/
│   │   ├── runtime.py
│   │   ├── registry.py
│   │   ├── prompts.py
│   │   ├── model_router.py
│   │   ├── context_builder.py
│   │   ├── validators.py
│   │   ├── reflection.py
│   │   └── guardrails.py
│   ├── tools/
│   │   ├── registry.py
│   │   ├── schemas.py
│   │   ├── permissions.py
│   │   ├── executor.py
│   │   ├── builtin/
│   │   └── adapters/
│   ├── mcp/
│   │   ├── client_manager.py
│   │   ├── server_registry.py
│   │   ├── transport.py
│   │   └── config.py
│   ├── memory/
│   │   ├── short_term.py
│   │   ├── long_term.py
│   │   ├── semantic.py
│   │   ├── extractor.py
│   │   ├── conflict_resolver.py
│   │   └── compressor.py
│   ├── rag/
│   │   ├── ingestion.py
│   │   ├── parsers.py
│   │   └── chunking.py
```

```

|   |   |—— embeddings.py
|   |   |—— retrievers.py
|   |   |—— hybrid.py
|   |   |—— reranker.py
|   |   |—— citations.py
|   |—— workflows/
|   |   |—— task_state.py
|   |   |—— graphs/
|   |—— models/
|   |—— schemas/
|   |—— services/
|   |—— repositories/
|   |—— db/
|   |—— core/
|   |—— main.py
|—— migrations/
|—— tests/
|—— pyproject.toml

```

不要创建尚未使用的目录和类。

只有当模块确实存在职责时，才创建对应文件。

五、LLM 与 Agent Runtime 改造

建立统一的模型调用层和 Agent Runtime。

不得允许业务代码在不同位置直接调用各个模型提供商 API。

1. Model Provider 抽象

至少统一以下能力：

- provider
- model
- API Endpoint
- API 配置
- 模型能力声明
- 超时
- 重试
- 流式输出
- 结构化工具调用
- Structured Output
- 最大输出长度
- 上下文窗口
- 推理参数
- 温度等生成参数
- Token 统计
- 估算成本
- 错误类型转换

- 请求追踪

第一版可以只完整支持一个模型提供商。

架构上应允许未来增加其他提供商，但不得为了多模型支持进行过度抽象。

2. Agent Runtime

Agent Runtime 至少支持：

- 接收用户目标
- 创建 Agent Run
- 构建上下文
- 调用 LLM
- 处理模型返回的结构化工具调用
- 验证工具参数
- 执行工具
- 将工具结果写回上下文和任务状态
- 判断是否继续执行
- 最大执行步数
- 最大工具调用次数
- 单工具超时
- 整体任务超时
- 用户取消
- 错误捕获
- 受控重试
- 最终回答
- Agent 事件流
- Trace ID
- Token 使用记录
- 运行结果持久化

Agent 循环应类似：

接收目标
→ 创建运行状态
→ 构建上下文
→ 调用模型
→ 模型直接回答或请求工具
→ 验证工具权限和参数
→ 必要时请求用户审批
→ 执行工具
→ 保存工具结果
→ 验证任务结果
→ 判断是否继续
→ 完成后输出最终结果

必须避免无限循环。

最大步骤数、最大重试次数和超时时间必须可配置。

六、工具调用系统

重构为统一的 Tool Registry 和 Tool Executor。

每个工具至少定义：

- 工具名称
- 工具描述
- 参数 Schema
- 返回值 Schema
- 工具类别
- 是否只读
- 风险等级
- 所需权限
- 超时时间
- 是否需要人工审批
- 可用 Agent 范围
- 是否支持取消
- 错误处理方式
- 日志脱敏策略

优先使用 Pydantic 或等价 Schema 验证参数和返回值。

工具风险分级

LOW:

时间查询、计算、知识库搜索、只读数据库查询、只读文件搜索

MEDIUM:

写入普通文件、更新任务、编辑普通记忆、创建非敏感数据

HIGH:

Shell 命令、删除文件、发送消息、部署项目、修改系统配置、数据库写操作、访问敏感文件、执行外部程序

HIGH 风险工具必须生成审批事件，用户批准后才能继续。

不得：

- 直接拼接用户输入形成 Shell 命令
- 允许任意路径文件访问
- 允许工具绕过权限模块
- 仅依赖模型判断操作是否安全

权限必须由程序代码强制执行。

七、MCP 改造

增加 MCP Client 管理模块，但不要把所有内部函数都改造成 MCP。

支持：

- MCP Server 注册
- MCP Server 编辑
- MCP Server 启用和禁用
- stdio 连接
- Streamable HTTP 连接
- 工具发现
- Resources 发现
- Prompts 发现
- 工具列表缓存
- 连接状态
- 健康检查
- 超时
- 重连
- 权限控制
- Agent 工具白名单
- MCP 调用日志
- 敏感配置保护
- 服务端错误隔离

MCP 配置不得写死在源码中，应通过数据库、环境变量或受控配置文件管理。

对于当前项目中已有插件、工具或扩展机制，请评估：

- 是否可以保留为 Function Tool
- 是否适合增加 MCP 适配层
- 是否应该独立为 MCP Server
- 是否与新工具系统重复
- 是否应该淘汰

遵循以下原则：

简单内部函数 → Function Tool

需要跨进程或跨应用复用 → MCP Tool

独立外部系统或远程服务 → MCP Server

八、上下文模块

新增独立的 ContextBuilder。

每次调用模型时，根据任务类型和 Token 预算动态组装：

- 系统提示词
- Agent 身份
- Agent 能力边界
- 安全约束
- 当前用户消息

- 最近对话
- 会话摘要
- 当前任务状态
- 已完成步骤
- 待执行步骤
- 工具调用结果
- 检索到的长期记忆
- RAG 检索结果
- 用户偏好
- 项目背景
- 引用来源

不得简单地把所有历史消息、所有记忆和所有文档全部发送给模型。

需要实现：

- Token 估算
- 不同模型的上下文预算
- 内容优先级
- 截断策略
- 工具输出裁剪
- 长日志摘要
- 相似内容去重
- 来源保留
- 内容类型预算
- 超预算降级方案

建议优先级：

当前用户目标
 > 系统安全约束
 > 当前任务状态
 > 必要工具结果
 > 高相关 RAG 内容
 > 高相关长期记忆
 > 最近对话
 > 历史摘要

上下文构建过程应可测试、可观测，并记录本次选择了哪些内容。

九、记忆模块

不得将所有聊天记录直接等同于记忆。

将记忆划分为以下类型。

1. 短期记忆

保存：

- 当前会话消息
- 当前任务状态
- 最近工具结果
- 当前执行计划
- 本轮临时变量

2. 长期结构化记忆

保存：

- 用户明确偏好
- 项目事实
- 重要决策
- 长期目标
- 已确认配置
- 历史任务结论
- 需要未来继续使用信息

推荐字段：

```
id
user_id
memory_type
key
value
source
source_id
confidence
importance
status
created_at
updated_at
expires_at
```

3. 语义记忆

使用 Embedding 和向量检索保存适合语义召回的历史信息。

语义记忆不得取代结构化记忆。

明确事实、开关状态、用户偏好和项目配置优先保存为结构化数据。

4. 记忆写入策略

不得保存每一句聊天。

记忆写入前必须判断：

- 是否具有长期价值
- 是否为明确事实
- 是否已经存在
- 是否与旧记忆冲突
- 是否只是模型推测
- 是否包含敏感信息
- 是否需要用户确认
- 是否应该设置过期时间

记忆模块需要支持：

- 查看记忆
- 搜索记忆
- 编辑记忆
- 删除记忆
- 禁用自动记忆
- 查看记忆来源
- 查看置信度
- 查看更新时间
- 处理冲突记忆
- 重新生成语义向量

不得把模型推测直接保存为高置信度用户事实。

十、记忆压缩和会话摘要

实现分层压缩：

原始消息
→ 会话局部摘要
→ 任务摘要
→ 项目摘要
→ 长期结构化记忆

摘要需要保留：

- 当前目标
- 已完成事项
- 未完成事项
- 重要决策
- 用户约束
- 文件名
- 模块名
- 错误信息
- 关键数字
- 已执行工具
- 下一步行动

建议采用结构化摘要：

```
{
  "goal": "",
  "decisions": [],
  "completed": [],
  "pending": [],
  "constraints": [],
  "important_facts": [],
  "errors": [],
  "files": [],
  "next_steps": []
}
```

压缩后不得立即删除全部原始内容。

应保留可配置的原始记录周期，并允许通过摘要追溯来源消息。

需要防止：

- 摘要丢失否定条件
- 摘要丢失关键数字
- 摘要把推测写成事实
- 摘要破坏时间顺序
- 摘要覆盖新的用户决定

十一、RAG 知识库

实现一个最小但完整、可扩展的 RAG 模块。

第一阶段至少支持：

- Markdown
- TXT
- PDF
- 常见代码文件
- 文档上传
- 文档解析
- 文档切块
- Embedding
- 知识库隔离
- 相似度检索
- 来源引用
- 文档删除
- 文档重新索引
- 索引状态
- 解析错误记录

1. 文档切块

不得只使用固定字符数切块。

根据文件类型选择策略：

- Markdown：按标题、章节和段落切分
- 代码：按类、函数、模块或语法结构切分
- 普通文本：按段落或语义切分
- PDF：按页面、标题和章节切分，并保留页码

每个 Chunk 应保存：

```
document_id
knowledge_base_id
content
chunk_index
source_name
page
section
language
content_hash
token_count
metadata
embedding_status
embedding_model
embedding_version
```

2. 检索流程

第一版建议实现：

```
用户问题
→ 查询规范化
→ 元数据过滤
→ 向量检索或全文检索
→ 去重
→ Token 预算裁剪
→ 返回相关片段和来源
```

第二阶段可增加：

```
关键词检索
+
向量语义检索
+
结果合并
+
```

去重
+
Reranker 重排序

3. RAG 工具化

将 RAG 封装为工具，例如：

```
search_knowledge_base  
get_document_chunk  
get_document  
list_knowledge_bases
```

Agent 应根据问题自行判断是否需要调用知识库。

不得每条消息都强制执行 RAG。

4. 引用

生成回答时应保留：

- 文档名称
- 章节
- 页码
- Chunk ID
- 相关原文片段
- 知识库名称

不得生成无法追溯的虚假引用。

十二、反思和验证机制

不要设计无限自我反思。

采用基于真实验证结果的受控重试：

```
执行  
→ 验证  
→ 成功则完成  
→ 失败则反馈明确错误  
→ 模型修正  
→ 最多重试一到两次
```

针对不同任务使用真实验证器：

- 代码：单元测试、编译、Lint、类型检查
- JSON：Schema 验证
- 文件修改：Diff 验证

- Shell：退出码、标准错误、超时
- RAG：引用是否支持答案
- 数据库操作：事务结果和数据库约束
- 多步骤任务：完成条件检查
- API 调用：状态码和响应 Schema

反思记录需要保存：

- 原执行步骤
- 失败原因
- 验证结果
- 修正建议
- 已重试次数
- 最终状态

反思不能绕过权限、安全规则和人工审批。

十三、LangChain 和 LangGraph 使用原则

不要默认引入 LangChain。

只有当当前项目确实需要其以下能力，并能显著降低维护成本时，才允许使用：

- 文档加载器
- Retriever
- 模型适配器
- Text Splitter
- Vector Store 适配器
- Reranker 集成

不得同时用 LangChain Agent 和自定义 Agent Runtime 管理同一执行循环。

LangGraph 只在以下需求明确存在时使用：

- 多步骤状态机
- 条件分支
- 循环执行
- 人工审批后继续
- 长任务暂停和恢复
- Checkpoint
- 多 Agent 协作
- 从指定节点恢复
- 多次检索和结果评估

如果当前阶段没有这些明确需求，先使用普通 Agent Runtime 实现。

十四、数据库设计

优先沿用并规范化当前项目的 MySQL 数据库。

不要仅为了追求某种技术栈而强制迁移到 PostgreSQL。

首先检查并记录：

- 当前 MySQL 具体版本
- 当前数据库部署方式
- ORM
- 数据库驱动
- 数据表结构
- 字符集
- 排序规则
- 数据迁移工具
- 全文检索能力
- 备份和恢复方式
- 当前数据量
- 当前是否保存用户、会话、消息、知识库、记忆和工具配置

除非发现 MySQL 无法满足明确的业务需求，或者存在不可解决的性能、兼容性和运维问题，否则保留 MySQL，并通过迁移脚本渐进式优化。

至少考虑以下实体：

```
users
agents
conversations
messages
agent_runs
run_steps
tool_calls
tool_approvals
tool_configs
mcp_servers
memories
knowledge_bases
documents
document_chunks
model_configs
audit_logs
```

数据库要求：

- 所有表具有明确主键
- 使用外键或应用层约束保证数据关系
- 用户数据必须隔离
- 会话数据必须隔离
- 记忆数据必须隔离
- 知识库数据必须隔离
- 所有核心业务表包含创建时间和更新时间
- 为高频查询字段建立合理索引
- 使用 utf8mb4 字符集
- 根据内容长度合理选择 TEXT、MEDIUMTEXT 或 LONGTEXT
- JSON 只用于适合半结构化存储的数据

- 不得使用 JSON 替代全部关系表设计
- 数据库修改必须通过迁移工具管理
- 提供升级脚本
- 提供回滚或恢复方案
- 明确软删除和硬删除策略
- 明确级联删除策略
- 密码必须使用安全哈希
- API Key 和 Token 不得明文存储
- 敏感配置使用加密存储或密钥管理服务
- 所有查询必须参数化
- 数据库写操作按需要使用事务
- 防止 SQL 注入
- 防止越权读取其他用户数据

向量检索与 RAG 存储策略

不要默认使用 pgvector。

根据当前 MySQL 版本、部署环境、数据规模和查询性能，选择以下方案。

方案 A：MySQL 加独立向量数据库

这是默认优先评估的方案。

MySQL 保存：

- 用户和权限
- 会话与消息
- Agent 运行记录
- 运行步骤
- 工具调用
- 工具审批
- MCP 配置
- 长期结构化记忆
- 知识库信息
- 文档信息
- Chunk 原文
- Chunk 元数据
- 向量记录与业务记录的映射关系

独立向量数据库可选择：

- Qdrant
- Milvus
- Weaviate
- 其他成熟并适合当前部署环境的向量数据库

向量数据库保存：

- 文档 Chunk 的 Embedding
- 语义记忆 Embedding
- 向量索引

- 向量检索元数据

推荐逻辑关系：

```
MySQL document_chunks.id
    ↓
向量数据库 point_id 或 metadata.chunk_id
    ↓
检索返回 chunk_id
    ↓
从 MySQL 查询原文、权限和来源
```

向量记录至少保存：

```
chunk_id
document_id
knowledge_base_id
user_id
embedding_model
embedding_version
content_hash
metadata
```

删除文档、知识库、用户或记忆时，必须同步清理 MySQL 和向量数据库中的关联记录。

必须处理一致性问题：

- MySQL 写入成功但向量写入失败
- 向量写入成功但 MySQL 事务失败
- 删除失败补偿
- 后台重新索引
- 索引任务状态
- 失败任务重试
- 孤立向量清理
- 文档版本更新
- Embedding 模型切换
- 重新生成向量
- 权限变化后的向量过滤

建议使用以下状态字段：

```
pending
processing
indexed
failed
deleted
```

不得假设 MySQL 和向量数据库可以进行同一事务。

需要通过任务状态、补偿机制和幂等操作保持最终一致性。

方案 B：使用 MySQL 原生向量能力

只有当前 MySQL 版本和实际部署环境明确支持所需的向量类型、距离函数、索引和查询性能时，才考虑直接在 MySQL 中保存 Embedding。

实施前必须实际验证：

- 支持的向量维度
- 支持的距离计算方式
- 是否支持余弦距离
- 是否支持点积
- 是否支持欧氏距离
- 是否具备适合当前数据量的向量索引
- 查询延迟
- 并发性能
- ORM 是否支持向量字段
- 数据库驱动是否支持
- Docker 环境和生产环境是否一致
- 迁移工具是否支持向量字段
- 备份与恢复是否支持向量数据

不得仅因为数据库存在 VECTOR 类型，就认定其适合生产级语义检索。

方案 C：第一版暂不使用向量数据库

如果当前知识库数据规模较小，可以先使用：

MySQL FULLTEXT 全文检索
+
元数据过滤
+
LLM 查询改写

完成基础 RAG。

后续文档数量、查询量或语义检索需求增长后，再增加独立向量数据库。

避免第一阶段引入过多基础设施。

混合检索

为后续混合检索预留统一 Retriever 接口：

MySQL 全文关键词检索
+
向量数据库语义检索
+

结果合并
+
去重
+
可选 Reranker

建议定义：

KeywordRetriever
VectorRetriever
HybridRetriever
MemoryRetriever

业务层只能调用统一 Retriever。

不得在 Agent Runtime、API 路由和页面代码中散落具体的 MySQL 或向量数据库查询。

数据迁移原则

如果项目已经存在 MySQL 数据：

1. 不得删除或覆盖现有数据库。
2. 迁移前备份数据库。
3. 验证备份能够恢复。
4. 分析现有表与目标表的映射关系。
5. 使用迁移脚本添加表和字段。
6. 数据转换必须可重复执行。
7. 使用幂等迁移逻辑。
8. 迁移过程记录成功、失败和跳过数量。
9. 迁移后校验用户、会话、消息、配置和知识库数量。
10. 新旧结构并行时，明确兼容期限。
11. 不允许通过无法追踪的手工 SQL 完成生产迁移。
12. 每个重要迁移提供恢复方案。

生成：

docs/database-design.md

文档至少说明：

- 为什么继续使用 MySQL
- 当前 MySQL 版本
- 当前数据库能力
- 表结构关系图
- 索引设计
- 事务边界
- 用户数据隔离方式
- RAG 向量存储方案

- MySQL 与向量数据库的一致性策略
- 备份恢复方案
- 数据迁移和回滚方案

十五、前端改造

在保留当前项目合理设计特色的基础上，整理 Agent 工作台交互。

页面至少展示：

- 对话内容
- 流式回答
- 当前 Agent
- 当前模型
- 运行状态
- 当前执行步骤
- 工具调用名称
- 工具参数摘要
- 工具执行结果
- 工具审批请求
- RAG 引用来源
- Token 使用量
- 估算成本
- 中止任务按钮
- 重新执行按钮
- 错误信息
- 会话列表
- 记忆管理
- 知识库管理
- MCP Server 管理
- 工具管理
- Agent 配置

不得展示模型隐藏的 Chain of Thought。

可以展示整理后的公开运行事件：

正在分析任务
正在检索知识库
找到 4 条相关资料
正在读取项目文件
等待确认执行命令
正在验证结果
任务已完成

前端和后端的单向流式通信优先使用 SSE。

只有确实需要双向实时通信时才使用 WebSocket。

十六、可观测性

每一次 Agent Run 生成唯一 Trace ID。

记录：

- 用户请求
- 用户 ID
- 会话 ID
- Agent
- 模型
- 开始时间
- 结束时间
- 每个步骤耗时
- 工具调用
- 工具错误
- Token 使用量
- 估算成本
- RAG 检索结果数量
- 检索耗时
- 重试次数
- 最终状态
- 取消原因
- 审批记录

日志必须结构化，并区分：

```
DEBUG
INFO
WARNING
ERROR
AUDIT
```

不得在日志中输出：

- API Key
- 密码
- 完整身份凭证
- 明文 Token
- 不必要的个人隐私
- 敏感文件全文
- 完整数据库连接密码

十七、安全要求

重点检查并修复：

- 任意文件读取
- 路径穿越

- Shell 命令注入
- SQL 注入
- SSRF
- 不安全文件上传
- 未授权工具调用
- MCP Server 配置注入
- API Key 泄露
- 数据库越权
- 不安全反序列化
- Prompt Injection
- RAG 文档中的恶意指令
- 工具返回内容中的恶意指令
- 前端 XSS
- CORS 配置不当
- 会话固定攻击
- 跨用户知识库访问
- 跨用户记忆访问
- 日志泄密

RAG 文档和工具返回内容必须视为不可信数据。

系统安全规则必须明确：

- 文档中的命令不是系统指令
- 工具返回内容不能覆盖系统安全规则
- MCP Server 返回内容不能直接获得高权限
- 高风险操作必须经过程序权限检查
- 模型不能自行授予权限
- 用户审批不能被工具结果伪造
- 数据库权限判断不能只依赖模型输出

十八、测试要求

至少补充以下测试。

单元测试

- ContextBuilder
- Token 预算
- Tool Registry
- Tool Schema
- 工具风险分级
- 工具权限判断
- 记忆提取
- 记忆去重
- 记忆冲突
- 会话摘要
- 文档切块
- RAG 检索
- 引用生成
- 数据隔离

- Retriever 接口
- 数据库 Repository

集成测试

- 用户消息到模型响应
- 完整工具调用循环
- 多轮工具调用
- 工具审批
- MCP 工具发现和调用
- RAG 文件上传
- 文档索引
- 知识库查询
- 长会话压缩
- Agent 任务取消
- 数据库迁移
- MySQL 与向量数据库映射
- 删除文档后的向量清理
- 失败索引重试

安全测试

- 路径穿越
- 命令注入
- SQL 注入
- 越权访问
- 恶意文件
- Prompt Injection
- RAG 注入
- 未授权高风险工具
- MCP 恶意参数
- 跨用户数据访问

不得只编写测试文件而不运行。

每次阶段性修改必须记录实际运行的命令和结果。

十九、实施阶段

按照以下阶段实施。

Phase 0: 审计与基线

- 阅读项目
- 生成审计文档
- 记录启动方式
- 尝试运行现有项目
- 建立基线测试
- 记录现有功能
- 不改变业务行为

Phase 1: 统一模型与 Agent Runtime

- 统一模型调用入口
- 增加结构化工具调用
- 增加 Agent Run
- 增加 Run Step
- 增加流式事件
- 增加最大步数
- 增加超时
- 增加任务取消

Phase 2: 工具和权限

- Tool Registry
- Tool Executor
- 参数 Schema
- 返回值 Schema
- 风险分级
- 人工审批
- 审计日志

Phase 3: 上下文和记忆

- ContextBuilder
- 会话存储
- 长期结构化记忆
- 语义记忆
- 记忆提取
- 记忆冲突
- 会话压缩

Phase 4: RAG

- 知识库
- 文档解析
- 文档切块
- Embedding
- MySQL 文档元数据
- 向量存储方案
- 检索
- 引用
- 删除和重新索引

Phase 5: MCP

- MCP Client Manager
- MCP Server 配置
- 工具发现
- stdio
- Streamable HTTP
- 权限
- 日志

- 健康检查

Phase 6: 复杂工作流

只有评估后确实需要，才引入 LangGraph：

- 状态图
- Checkpoint
- 人工审批恢复
- 失败重试
- 多次 RAG 检索
- 多 Agent

Phase 7: 前端与整体体验

- Agent 运行过程
- 工具审批
- 知识库管理
- 记忆管理
- MCP 管理
- 工具管理
- 错误追踪
- 运行历史

每个阶段都必须：

1. 写明本阶段目标。
2. 列出准备修改的文件。
3. 实施代码修改。
4. 运行测试。
5. 运行静态检查。
6. 运行构建或启动验证。
7. 汇报实际结果。
8. 更新文档。
9. 生成清晰的 Git Diff 摘要。
10. 确保项目仍可启动。
11. 不得在测试失败时声称阶段已经完成。

二十、最终交付物

最终至少提供：

```
docs/
├── modernization-audit.md
├── target-architecture.md
├── migration-plan.md
├── agent-runtime.md
├── tool-system.md
├── context-design.md
├── memory-design.md
├── rag-design.md
```

```
|—— mcp-design.md
|—— database-design.md
|—— security-model.md
|—— testing-guide.md
|—— deployment-guide.md
```

同时更新：

- README
- 环境变量示例
- 依赖文件
- 依赖锁文件
- Dockerfile
- Docker Compose
- MySQL 初始化或迁移说明
- 向量数据库配置
- 开发启动命令
- 生产部署命令
- 架构图
- API 文档
- 故障排查文档

二十一、开始执行时的顺序

现在先不要进行大规模代码修改。

请依次完成：

1. 扫描整个仓库。
2. 阅读 README、依赖文件和配置文件。
3. 尝试按照当前文档启动项目。
4. 列出当前技术栈和关键依赖版本。
5. 定位模型调用、工具调用、记忆、上下文、RAG 和数据库入口。
6. 画出当前系统架构和数据流。
7. 找出最严重的十个架构或安全问题。
8. 输出保留、封装、替换和删除清单。
9. 输出目标架构。
10. 输出分阶段迁移计划。
11. 生成 `docs/modernization-audit.md`。
12. 生成 `docs/migration-plan.md`。
13. 生成 `docs/target-architecture.md`。
14. 完成以上内容后，再从 Phase 1 中风险最低、最容易验证的改造开始。

遇到信息不完整时，优先通过阅读代码、配置、数据库迁移、提交记录、测试和实际运行结果自行判断，不要凭空假设。

每个结论必须引用具体的文件、类、函数、表结构或配置位置。

所有改造都必须以：

- 能够运行
- 能够测试
- 能够验证
- 能够回滚
- 不丢失数据
- 不降低安全性

为最终标准，而不是只追求目录结构和技术名词看起来先进。